

NextAgent

Plataforma de Creación y Despliegue de Agentes de IA Verificables

Equipo	Oscar David Rojas Bedoya · Yesid Alejandro Peláez Posada
Proyecto	Beca IA Ser ANDI — Reto EAFIT × Verana
Repositorio	github.com/alejopp/agent-challenge-eafit
Plataforma	persona-ai.team-g.teams.eafit.testnet.verana.network
Agente demo	alice.agents.team-g.teams.eafit.testnet.verana.network
Fecha	Mayo 2026

Tabla de Contenidos

1.	Contexto del Reto
2.	Definición del Problema
3.	Justificación de la Solución
4.	Metodología de Desarrollo
5.	Arquitectura Técnica
6.	Flujo de Datos
7.	Bases de Datos Utilizadas
8.	APIs, Modelos y Servicios
9.	Componentes de IA
10.	Resultados Obtenidos
11.	Dificultades Encontradas
12.	Recomendaciones
13.	Conclusiones

1. Contexto del Reto

1.1. Programa y Organización

El proyecto se desarrolla en el marco del programa **Beca IA Ser ANDI**, una iniciativa conjunta entre **NODO EAFIT** y la **Verana Foundation**, orientada a formar talento técnico universitario en tecnologías emergentes de inteligencia artificial e identidad digital descentralizada. El reto propuesto — *Agentes IA Verificables con Hologram* — tiene como trasfondo el ecosistema tecnológico de Verana y su aplicación de mensajería Hologram.

1.2. El Ecosistema Verana / Hologram

- **Verana**: capa de confianza abierta fundamentada en los estándares W3C de DIDs y Credenciales Verificables, que permite identificar actores digitales de forma criptográfica sin intermediarios.
- **Hologram**: aplicación de mensajería que actúa como navegador de agentes IA verificables, mostrando al usuario quién opera el agente respaldado por una credencial verificable.
- **VS Agent** (Verifiable Service Agent): middleware de identidad que gestiona el DID propio del agente, establece canales DIDComm cifrados punto a punto y recibe credenciales de servicio.

1.3. Concepto de Persona AI Agent

Un **Persona AI Agent** representa a una persona real y actúa en su nombre dentro del ecosistema Hologram. Cada agente posee una identidad DID verificable, una Credencial Verificable de servicio emitida por EAFIT, un canal DIDComm cifrado y capacidades extendidas mediante servidores MCP.

Ejemplo: un plomero puede tener un agente que gestione su calendario. Los clientes se conectan vía Hologram para agendar una intervención, confirmando previamente la legitimidad del agente.

2. Definición del Problema

2.1. Problema Central

Los chatbots y agentes de IA operan actualmente en plataformas centralizadas sin mecanismos de confianza verificable, lo que genera tres problemáticas fundamentales:

Identidad no verificable. No existe mecanismo que confirme quién opera un bot, qué políticas sigue ni si actúa legítimamente. Cualquier actor puede suplantar la identidad de una empresa o profesional sin posibilidad de detección.

Confianza ciega del usuario. Las personas interactúan con chatbots sin poder verificar su autenticidad, quedando expuestas a desinformación, fraude o phishing. La confianza se otorga implícitamente, sin base criptográfica.

Barreras técnicas de despliegue. Crear un agente con identidad descentralizada requiere conocimientos avanzados en Docker/Kubernetes, protocolos DIDComm, Credenciales Verificables W3C, Helm charts y pipelines CI/CD.

2.2. Brecha Identificada

El ecosistema Verana/Hologram resuelve la confianza criptográfica, pero el proceso de incorporarse seguía siendo complejo y manual. **No existía una plataforma que abstraiera completamente este proceso**, permitiendo que cualquier persona publicara su agente verificable sin escribir código.

3. Justificación de la Solución

3.1. Propuesta de Valor

NextAgent es una plataforma web que elimina la complejidad técnica del despliegue de agentes IA verificables, entregando a cualquier usuario —sin conocimientos de programación— la capacidad de crear, configurar y publicar su propio Persona AI Agent en el ecosistema Hologram/Verana.

3.2. Automatización del Pipeline Técnico

1. Generación dinámica de Helm values y agent-pack.yaml por agente
2. Creación de schema PostgreSQL dedicado en la base de datos compartida
3. Despliegue del VS Agent en Kubernetes mediante helm upgrade --install
4. Sondeo del endpoint /v1/agent hasta confirmar disponibilidad
5. Emisión automática de la credencial de servicio verificable (EAFIT)
6. Generación del QR de conexión para usuarios finales de Hologram

3.3. Comparativa con el Enfoque Manual

Tarea	Sin NextAgent	Con NextAgent
Configurar Helm chart	1–2 horas (experto)	Automático
Desplegar VS Agent en K8s	30–60 min (experto)	1 clic
Emitir credencial Verana	30–60 min (experto)	Automático
Usuario no técnico (total)	Inviabile	5–10 minutos

4. Metodología de Desarrollo

4.1. Enfoque General

Desarrollo ágil e iterativo priorizando la entrega de valor funcional en cada ciclo. Las decisiones técnicas se tomaron de forma incremental, validando cada componente antes de avanzar. Se utilizó **Windsurf IDE** con asistencia de IA (Cascade) como entorno principal.

4.2. Stack Tecnológico

Capa	Tecnologías
Frontend	React 18, Vite, React Router v6
Backend	Node.js 20, Express.js, Passport.js, JWT, Multer
Infraestructura	Kubernetes (namespace team-g), Helm 3, Docker, Docker Hub
CI/CD	GitHub Actions
Red Verana	VS Agent, DIDComm/WSS, Credenciales Verificables W3C
Shared Infra	PostgreSQL, Redis, OpenAI GPT-4o-mini
MCP	Open-Meteo API, Wikimedia API, Google Calendar API

4.3. Fases de Desarrollo

Fase 1 — Exploración del ecosistema

Comprensión del VS Agent y protocolo DIDComm: fork del repositorio oficial, configuración local con Docker Compose, personalización del agent-pack.yaml, incorporación de documentos RAG y exposición vía ngrok. Resultado: chatbot funcional en Hologram con identidad verificable local.

Fase 2 — Despliegue en Kubernetes

Traslado a infraestructura productiva: configuración del kubeconfig, creación del Helm chart, configuración de Ingress con dominio público. Resultado: agente Alice operativo y accesible desde Hologram con credencial EAFIT.

Fase 3 — Plataforma web

Desarrollo full-stack iterativo: autenticación JWT + Google OAuth, pipeline de publicación automatizado, QR de conexión, integraciones MCP (clima, Wikipedia), CI/CD con GitHub Actions y limpieza automática de locks Helm.

4.4. Desafíos Técnicos y Resolución

Desafío	Solución aplicada
Timeout en credencialización (DID tarda 3–7 min)	Bucle de sondeo con backoff exponencial sobre /v1/agent
Lock de Helm en despliegues fallidos	Step previo en CI/CD con helm rollback --force
QR de invitación OOB no directo	QR redirige a página pública del VS Agent (solución en curso)
TLS autofirmado rechazando conexiones internas	NODE_TLS_REJECT_UNAUTHORIZED=0 exclusivo para testnet
Google OAuth fallando en producción	Migración a rutas relativas + configuración APP_URL en Helm values

4.5. Estado Final del Proyecto

Funcionalidad	Estado
Registro y login (local + Google OAuth)	■ Funcional
CRUD de bots con imagen y documentos RAG	■ Funcional
Publicación automática en Kubernetes vía Helm	■ Funcional
Emisión de credencial verificable (EAFIT)	■ Funcional
QR de conexión visible en la plataforma	■ Funcional
Integración MCP — Clima (Open-Meteo)	■ Funcional
Integración MCP — Wikipedia (Wikimedia)	■ Funcional
Conexión verificable desde app Hologram	■ Funcional
CI/CD con limpieza automática de lock Helm	■ Funcional
Integración MCP — Google Calendar	■ En ajuste

Funcionalidad	Estado
QR de invitación OOB directa	■ En ajuste

5. Arquitectura Técnica

La arquitectura de NextAgent se fundamenta en un modelo de microservicios distribuido sobre **Kubernetes** en el namespace team-g, priorizando la escalabilidad de agentes individuales con reutilización de componentes críticos de infraestructura.

5.1. Componentes del Sistema

Backend (Node.js + Express). Centro de mando: gestiona autenticación, persistencia de configuraciones y orquestación de comandos helm/kubectl para el despliegue de infraestructura.

Frontend (React + Vite). SPA moderna que abstrae la complejidad técnica del cluster en una experiencia de usuario fluida.

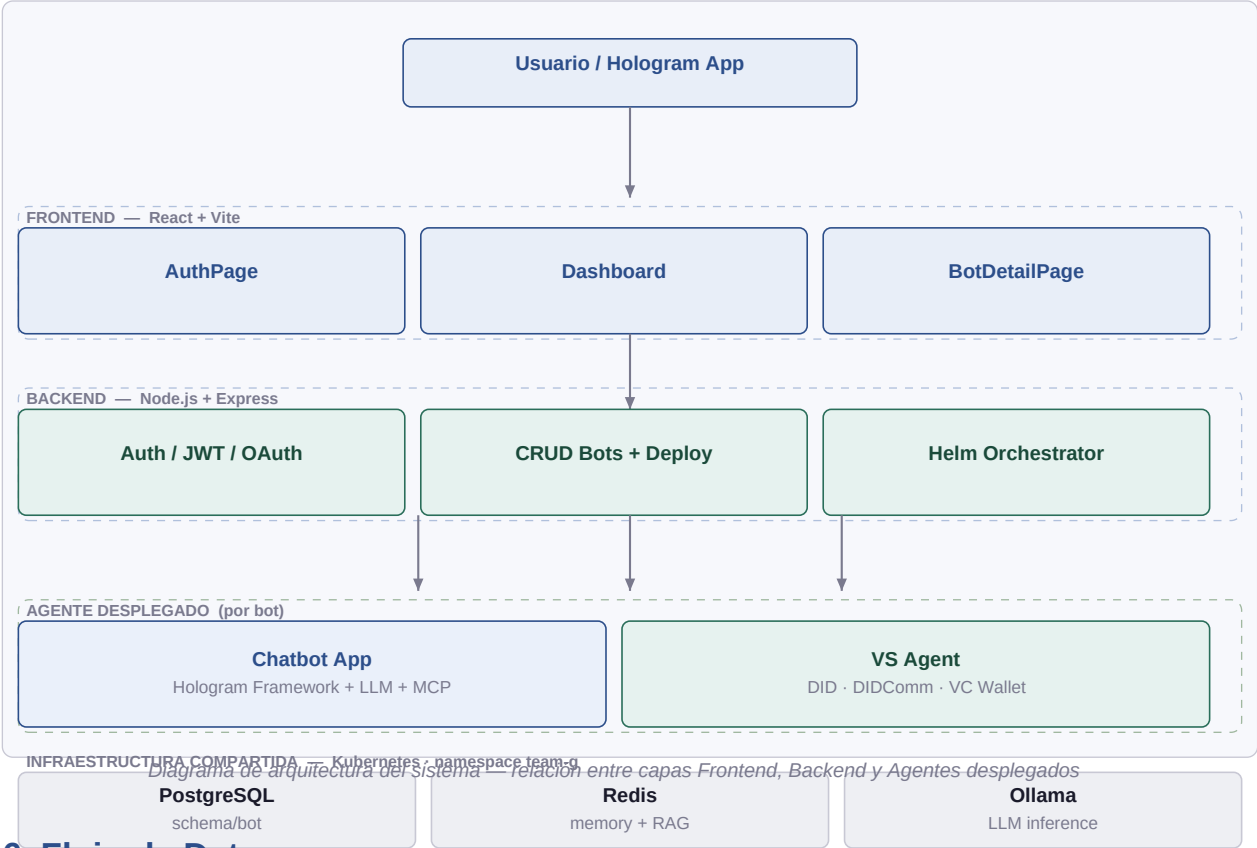
Chatbot App (por agente). Motor de ejecución basado en el framework Hologram; procesa conversaciones usando LLM + RAG + MCP.

VS Agent (por agente). Middleware de identidad: gestiona DID propio, comunicaciones DIDComm cifradas y wallets de credenciales.

5.2. Infraestructura Compartida

Componente	Rol
PostgreSQL compartido	Una instancia, schema dedicado por bot para aislamiento de datos.
Redis compartido	Memoria de ventana conversacional y vector store para RAG.
Ollama compartido	Servicio de inferencia LLM centralizado, API compatible con OpenAI.

Diagrama de Arquitectura



6. Flujo de Datos

El ciclo de vida de la información sigue un proceso lineal desde la creación del bot hasta la interacción con el usuario final:

1	Captura de Configuración	El usuario ingresa la Persona del bot y sube documentos. Los archivos se almacenan temporalmente y la metadata persiste en SQLite.
2	Preparación de Despliegue	El backend transforma los datos en archivos YAML estructurados (agent-pack.yaml y values.yaml).
3	Ejecución en Kubernetes	El backend invoca Helm para crear los recursos (Deployments, Services, ConfigMaps) en el cluster.
4	Autenticación del Agente	Una vez activo el bot, el sistema consulta su DID público y solicita una Credencial Verificable a la organización raíz (EAFIT).
5	Interacción DIDComm	El usuario escanea el QR generado, estableciendo un canal cifrado entre su app Hologram y el VS Agent desplegado para una conversación segura y verificada.

7. Bases de Datos Utilizadas

Motor	Capa	Función
SQLite	Capa de Aplicación	Gestión interna de NextAgent: usuarios y sesiones (JWT), perfiles de bots creados, registro de estados de despliegue (Pendiente, Publicado, Error).

Motor	Capa	Función
PostgreSQL	Capa de Persistencia de Agentes	Almacena la lógica persistente de los agentes publicados. Cada bot utiliza un usuario y schema dedicado para logs de auditoría, historiales e metadatos de configuración del VS Agent.
Redis	Capa de Memoria y RAG	Memoria de ventana conversacional (últimos turnos de conversación) y vector store para indexación semántica de documentos subidos por el usuario (PDF/TXT).

8. APIs, Modelos y Servicios Implementados

Categoría	Servicio / Modelo	Descripción
LLM	GPT-4o-mini (OpenAI)	Modelo principal para razonamiento y uso de herramientas.
LLM	Llama 3.2 (Ollama)	Alternativa local para inferencia en el cluster.
MCP — Clima	Open-Meteo API	Datos geográficos y meteorológicos en tiempo real.
MCP — Wikipedia	Wikimedia API	Resúmenes enciclopédicos y búsqueda de conocimiento.
MCP — Calendario	Google Calendar API	Gestión de eventos y disponibilidad (en ajuste).
Identidad	Verana Foundation	Resolución de DIDs y validación de confianza en la red.
Auth	Google OAuth 2.0	Autenticación federada de usuarios vía Google Cloud.

9. Componentes de Inteligencia Artificial

La inteligencia de cada agente converge en tres tecnologías complementarias:

1	Persona Engine (Prompting) Transforma el lenguaje natural en instrucciones de sistema rigurosas. El prompt define la personalidad del bot, sus límites éticos, estilo de respuesta y prioridades operativas. Se genera dinámicamente para cada bot en el momento del despliegue.
2	Retrieval-Augmented Generation (RAG) Permite que el agente acceda a conocimiento privado. Los documentos subidos (PDF/TXT) se fragmentan, convierten en embeddings y almacenan en Redis. Durante una consulta, el bot busca en estos vectores la información más relevante antes de formular la respuesta.
3	Tool Use — Integración MCP A diferencia de los chatbots estáticos, los agentes de NextAgent son dinámicos. El LLM decide autónomamente cuándo invocar herramientas externas (clima, Wikipedia) para responder con datos reales en lugar de depender exclusivamente de su conocimiento de entrenamiento.

10. Resultados Obtenidos

< 5 min	Tiempo de despliegue de un agente verificable (antes: ~2 horas)
100 %	Agentes publicados que obtienen el sello Verificado en Hologram
Multi-LLM	Soporte para GPT-4o-mini (OpenAI) y Llama 3.2 (Ollama) de forma transparente
No-code	Cualquier usuario no técnico puede publicar su propio agente verificable

11. Dificultades Encontradas

#	Problema	Fecha	Impacto	Solución
1	Namespace eliminado accidentalmente	27/04	Alto	Solicitud a EAFIT vía Discord; restauración el mismo día.
2	Timing de credencialización Verana (3–7 min)	1–5/05	Alto	Bucle de sondeo con backoff exponencial; notificación al usuario en tiempo real.
3	Lock de Helm en despliegues fallidos	3–4/05	Alto	Step de limpieza automática (helm rollback) en GitHub Actions.
4	Google OAuth fallando en producción	5–6/05	Medio	Migración a rutas relativas + variable APP_URL en Helm values.
5	QR no funcional directamente con Hologram	5–6/05	Bajo-Med.	QR redirige a página del VS Agent; solución definitiva en desarrollo.
6	Agente sin respuesta durante demo	6/05	Alto	Error en integración MCP Wikipedia; corregido el 7/05.
7	Descarga de logo fallaba silenciosamente	6–7/05	Medio	Manejo explícito de errores en script bash; skip si imagen falla.
8	TLS autofirmado rechazando conexiones internas	Persistent.	Bajo	NODE_TLS_REJECT_UNAUTHORIZED=0 controlado para testnet.
9	Permisos de GitHub bloqueaban a Oscar	1/05	Bajo	Configuración de clave SSH + ajuste de permisos en el repositorio.
10	Chatbot respondía en idioma incorrecto	3/05	Medio	Migración de Groq a OpenAI GPT-4o-mini como modelo base.

12. Recomendaciones

Para un agente de calidad

- **Prompt específico:** incluir tono de voz, idioma, temas permitidos, temas restringidos y comportamiento ante respuestas desconocidas.
- **Documentos relevantes:** subir un PDF bien estructurado con preguntas frecuentes mejora significativamente la calidad de las respuestas vía RAG.

— **Prueba antes de compartir:** conectarse al agente desde Hologram y verificar su comportamiento antes de distribuir el QR.

Para la publicación

- No usar caracteres especiales en el nombre del agente (evitar ñ, tildes, símbolos).
- No cerrar el navegador durante la publicación; esperar el mensaje de éxito.
- La primera publicación es la más lenta (5–7 min); las republicaciones son más rápidas.

13. Conclusiones

1. Democratización de la Identidad Digital

El proyecto demuestra que es posible abstraer la complejidad de los protocolos DID y Credenciales Verificables mediante interfaces intuitivas, permitiendo que el valor de la red Verana llegue a usuarios finales y profesionales sin conocimientos técnicos.

2. Eficiencia en el Despliegue de Agentes

La automatización integral del pipeline —desde la configuración hasta el despliegue en Kubernetes y la emisión de credenciales— reduce drásticamente las barreras de entrada. Lo que antes requería intervención experta ahora es un proceso reproducible, escalable y auditable en minutos.

3. Valor del Ecosistema Académico-Industrial

La colaboración entre NODO EAFIT y Verana Foundation proporciona un entorno ideal para la experimentación con tecnologías de vanguardia, aplicando conocimientos de orquestación, desarrollo full-stack e IA en un caso de uso real con impacto directo en el futuro de los agentes inteligentes verificables.
